Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

Linux цөмийн модулиар процессын жагсаалт ба холбоосыг дүрслэх

Linux Kernel Module for Listing Tasks

Бие даалт №1

Судалгаа, хэрэгжилт, туршилтын тайлан

Шалгасан багш: Г.Ганбат

Багийн дугаар: IV-2-3

Гүйцэтгэсэн оюутнууд: А.Тэмүүлэн /B242270063/

Ч.Балгандамба /B232270142/

Д.Азмаа /B241940101/

Н.Билэгт-Очир /B231910009/

Т.Түвшинтөр /B190910820/

Б.Өсөхбаяр /B210930901/

Улаанбаатар хот

2026 он

Гарчиг

Хураангуй **1**

1 Удиртгал **1**

2 Сурах бичгийн шаардлага **1**

3 Онолын үндэс **2**

4 Системийн зохиомж **2**

5 Хэрэгжилт **2**

6 Туршилт ба UI дүрслэл **3**

7 Туршилтын арга **3**

8 Үр дүн **4**
8.1 Процесс ба thread-ийн граф 5
8.2 Төлөвийн өөрчлөлт 6

9 Хэлэлцүүлэг **7**

10 Дүгнэлт **7**

Ашигласан материал **7**

Хураангуй

Linux системийн процесс, thread-ийн мэдээллийг kernel module-оор уншиж, холбоосыг графаар харуулах TaskGraph програм хэрэгжүүлэв. Модуль нь PID, төлөв, нэрийг LINEAR болон DFS дарааллаар хэвлэж, `/proc/taskgraph` файлаар шинэ хэмжилт өгнө. UI-аас дэд мод сонгох, thread шүүх, дарааллаар үзэх, мэдээлэл хадгалах боломжтой. Python-ийн 23, бодит Linux kernel-ийн 12 шалгалт нийт 35/35 тэнцсэн.

1. Удиртгал

Зорилго нь Linux-ийн процессын бүтэц, эцэг-хүүхэд холбоо болон DFS аргыг бодит өгөгдөл дээр судлах юм. Процесс үүсэх, зогсох, дуусах өөрчлөлтийг граф, хүснэгтээр ажиглана. Хэрэгжилт нь C kernel module, Python parser, Tkinter UI гэсэн гурван хэсэгтэй.

2. Сурах бичгийн шаардлага

Operating System Concepts сурах бичгийн 3-р бүлгийн Programming Project 3 нь task-уудыг шугаман болон гүний хайлтаар жагсаахыг шаарддаг [1].

Хүснэгт 1: Үндсэн шаардлага ба хэрэгжилт

Шаардлага	Хэрэгжилт
Шугаман жагсаалт	`for_each_process()` ашиглаж process leader бүрийн PID, state, comm-ийг kernel log-д хэвлэнэ.
DFS жагсаалт	`init_task`-аас эхэлсэн модыг kernel дотор стекээр туулж, дараалал ба гүнийг гаргана.
Зөв ажиллагааг шалгах	Process жагсаалтыг `ps -el`, thread-үүдийг `ps -eLf`-тэй харьцуулж болно.
Task graph UI	Parent-child болон process-thread холбоосыг дүрсэлж, дахин уншихад шинэчилнэ.
Өөрчилж турших	Root PID, thread шүүлт, харагдах тоо, шинэчлэх хугацааг өөрчилнө. Буруу оролтыг мэдээлнэ.

Номын `children/sibling` холбоосыг шууд рекурсээр дагахын оронд хамгаалалттай хуулсан parent ID-аас мод байгуулсан. Ингэснээр өөрчлөгдөж буй kernel жагсаалтыг түгжээгүй дагахгүй, kernel stack дээр рекурс ашиглахгүй.

3. Онолын үндэс

Linux-ийн `task_struct` нь task-ийн PID, төлөв, нэр болон холбоосыг агуулна. Нэг процесс олон thread-тэй байж болно.

- **PID**: task-ийн дугаар; **TGID**: thread group-ийн дугаар. Process leader-ийн PID ба TGID ижил.
- **PPID**: эцэг процессын дугаар. Графт process-ийг PPID-тай, нэмэлт thread-ийг өөрийн TGID-тай холбоно.
- **LINEAR**: process жагсаалтыг дарааллаар унших арга.
- **DFS**: эхлээд эцэг зангилаа, дараа нь хүүхдийн дэд модыг бүрэн туулна.

Төлөвийн тэмдэг: R — ажиллаж байгаа эсвэл бэлэн; S — унтаж байгаа; I — idle kernel thread; D — uninterruptible sleep; T/t — зогсоосон; Z/X — zombie/dead.

4. Системийн зохиомж

Өгөгдөл `task_struct` → kernel snapshot → `/proc/taskgraph` → Python parser → граф, хүснэгт гэсэн урсгалаар дамжина.

Хүснэгт 2: Компонентуудын үүрэг

Компонент	Үүрэг
`taskgraph.c`	Task-уудын мэдээллийг хуулж, parent индекс, DFS дараалал тооцно.
`model.py`	TSV өгөгдлийг шалгаж, дэд мод болон thread шүүлт хийнэ.
`gui.py`, `cli.py`	Нэг өгөгдлийг график болон командын мөрөөр үзүүлнэ.
`workload.py`	Өөр өөр гүн, салаалалттай туршилтын процессын мод үүсгэнэ.

Snapshot бүр `pid, ppid, tgid, parent, state, kind, linear, dfs, depth, orphan, comm` баганатай. `count, cap, truncated` нь хэмжилт бүтэн эсэхийг мэдээлнэ.

5. Хэрэгжилт

`tg_capture()` нь санах ойг урьдчилан хуваарилж, RCU хамгаалалттай үед task-ийн утгуудыг хуулна [2]. Lock-оос гарсны дараа parent ID-аар мод байгуулж, `tg_tree()` функцээр DFS гүйцэтгэнэ. Модулийг ачаалахад LINEAR ба DFS жагсаалт kernel log-д гарна. `seq_file` нь нэг нээлттэй файлаас хэсэгчлэн уншихад ижил snapshot ашиглана [3]. Дахин нээх бүр шинэ өгөгдөл авна. `max_tasks` нь 16–16384, анхны утга 4096. Хязгаар хүрвэл `truncated=1` гэж мэдээлнэ. Parser нь формат, PID давхардал, дараалал, төлөв болон оролтын хэмжээг шалгана.

6. Туршилт ба UI дүрслэл

Зүүн хэсэгт task graph, баруун хэсэгт хүснэгт харагдана. Тасралтгүй сум нь process parent-child холбоо, тасархай сум нь thread group-ийн гишүүнчлэл юм. Зангилааг сонгоход доод мөрөнд дэлгэрэнгүй гарна.

Хүснэгт 3: Өөрчилж турших удирдлагууд

Удирдлага	Үйлдэл
DFS / LINEAR	Task үзүүлэх дарааллыг солино.
Root PID / Threads	Дэд мод сонгож, нэмэлт thread-үүдийг харуулна эсвэл нууна.
Зангилааны хязгаар	1–2000 хүртэл task харуулна; бүтэн snapshot-ийг өөрчлөхгүй.
Шинэчлэх / Авто шинэчлэлт	Эх үүсвэрийг дахин уншина. Интервал 500–10000 ms.
Өмнөх / Дараах / Тоглуулах	Нэг snapshot дотор traversal-ийг алхмаар харуулна.
Файл нээх / Хадгалах / Reset	Snapshot импортлох, экспортлох, view тохиргоо сэргээх.

Kernel горим нь бодит модулиас өгөгдөл авна. **File горим** нь хадгалсан хэмжилтийг харуулна. **Fixture горим** нь алгоритм шалгах жишээ өгөгдөлтэй. Эх үүсвэрийг дэлгэцэд ялгаж тэмдэглэсэн. File горимын автомат шинэчлэлт нь ижил файлыг дахин уншина.

7. Туршилтын арга

Хүснэгт 4: Туршсан орчин

Хэсэг	Орчин
Build	Ubuntu 24.04, GCC 13.3.0, Linux 6.8.0-31 headers.
Kernel ажиллагаа	QEMU 8.2.2 TCG, 2 vCPU, 512 MiB RAM, Linux 6.8.0-31-generic, initramfs.
UI, автомат тест	Python 3.12, Tk 8.6, unittest, Xvfb.

Python тестээр оролт шалгах, LINEAR/DFS, дэд мод, thread шүүлт, импорт/экспорт, буруу оролт болон 5000 гүн модыг шалгав. Kernel тестээр модулийг ачаалж, процесс ба thread үүсгэн `before`, `created`, `stopped`, `after`, `capped` үеийн snapshot авсан. C workload нь 5 process, нэмэлт 2 thread буюу 7 task үүсгэнэ. Root-д SIGSTOP өгөхөд T төлөв болох, дуусгахад бүх хүүхэд цэвэрлэгдэхийг шалгасан. Үр дүнг `docs/evidence/` дотор log, JSON, TSV хэлбэрээр хадгалсан.

8. Үр дүн

Python-ийн **23/23 тест**, kernel-ийн **12/12 шалгалт** тэнцэв. Модуль амжилттай хөрвөж, Linux 6.8.0-31 дээр ачаалагдсан.

Хүснэгт 5: Бодит kernel дээрх шалгалтын үр дүн

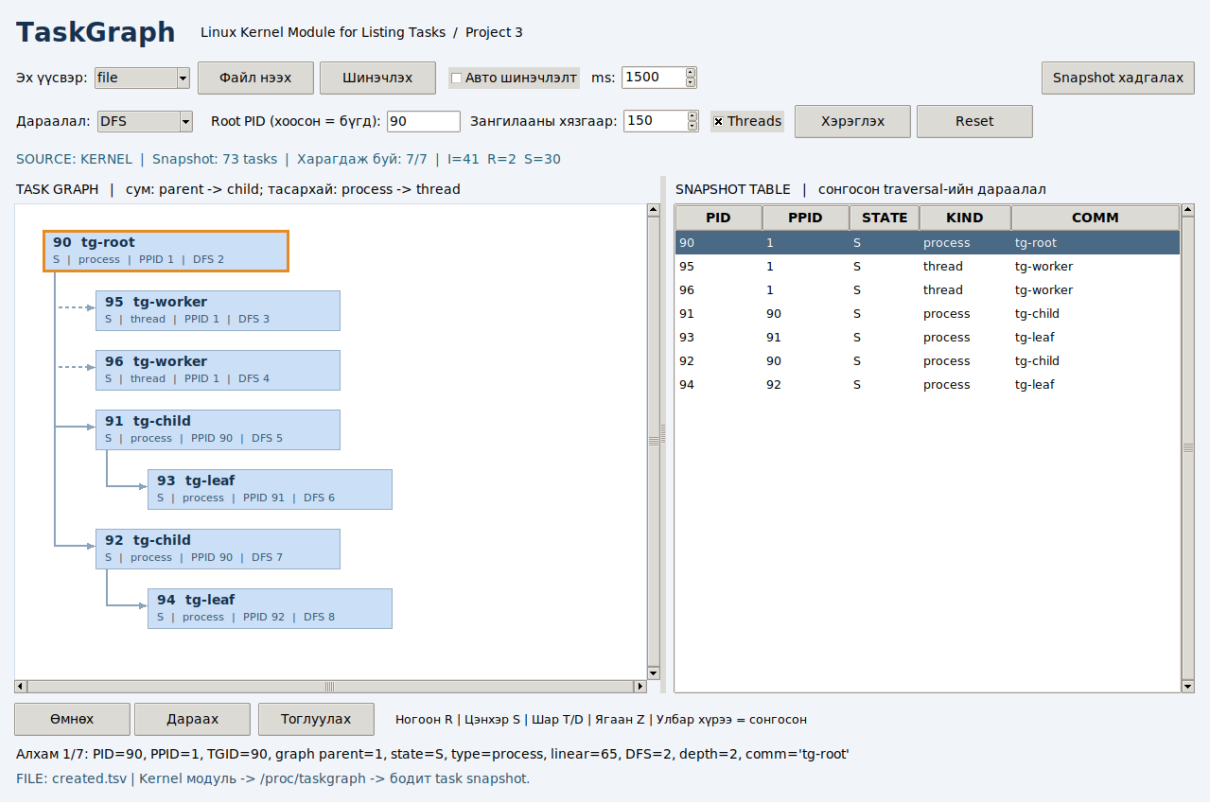
Код	Ажигласан үр дүн	Төлөв
K1	Модуль ачаалагдав; `insmod` exit=0.	Тэнцсэн
K2	Kernel эх үүсвэртэй 66 task; PID 0 багтсан.	Тэнцсэн
K3	LINEAR болон DFS log хоёулаа хэвлэгдсэн.	Тэнцсэн
K4	73 task-ийн DFS rank давхцаагүй; parent нь child-ээс өмнө.	Тэнцсэн
K5	Root PID 90-ийн дэд мод 7 task-тай.	Тэнцсэн
K6	PID 95, 96 thread-үүд TGID 90-тай холбогдсон.	Тэнцсэн
K7	SIGSTOP дараа root-ийн state=T болсон.	Тэнцсэн
K8	Workload дууссаны дараа `tg-*` task үлдээгүй.	Тэнцсэн
K9	Модуль буухад `/proc/taskgraph` устсан.	Тэнцсэн
K10	`max_tasks=1` буруу утгыг хүлээж аваагүй.	Тэнцсэн
K11	`max_tasks=16`: count=16, truncated=1.	Тэнцсэн
K12	Дахин ачаалах, буулгах үйлдэл амжилттай.	Тэнцсэн

Хүснэгт 6: UI болон өгөгдлийн боловсруулалтын шалгалт

Туршилт	Хүлээгдсэн ба гарсан үр дүн
Жишээний DFS	0, 1, 10, 11, 20, 21, 2.
Root PID=10	10, 11, 20, 21 гэсэн 4 task харагдсан.
Threads нуух	PID 11 нуугдсан.
Байхгүй PID, буруу хязгаар	Ойлгомжтой алдаа гарсан.
Импорт/экспорт	Snapshot-ийн task өгөгдөл өөрчлөгдөөгүй.

Kernel тестийн дэлгэрэнгүй нь `kernel_results.json`, serial log нь `qemu-serial.log`, Python тестийн гаралт нь `docs/unit-tests.log` файлд бий. PID нь дахин ажиллуулахад өөрчлөгдөх тул тест root PID-г тухайн run-оос уншдаг.

8.1. Процесс ба thread-ийн граф



Зураг 1: Kernel-ийн created snapshot-ийг UI-д нээсэн байдал.

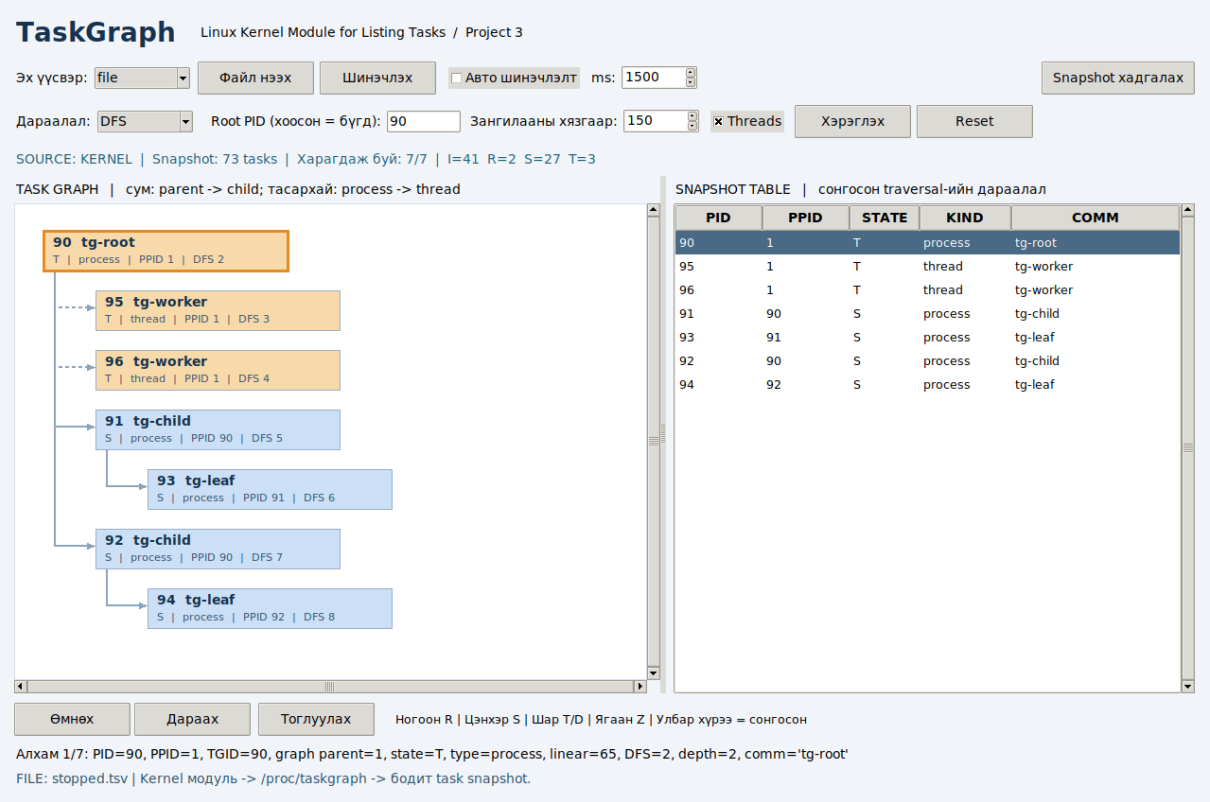
Root PID 90-ийн дэд модонд 7 task байна. PID 95, 96 нь thread тул тасархай сумаар холбогдсон. PID 91, 92 нь root-ийн хүүхэд; PID 93, 94 нь тэдгээрийн хүүхэд юм.

Хүснэгт 7: Зурагт харагдаж буй task-ууд

PID	Графын parent	Төрөл	Нэр
90	1	Process	`tg-root`
95, 96	90	Thread	`tg-worker`
91, 92	90	Process	`tg-child`
93, 94	91, 92	Process	`tg-leaf`

Зургийн өгөгдлийг QEMU дахь бодит Linux kernel module-оос авч, `created.tsv` файлаар UI-д импортолсон. Графын холбоос, хүснэгтийн мөрүүд нэг snapshot-д тулгуурлана.

8.2. Төлөвийн өөрчлөлт



Зураг 2: SIGSTOP дараах snapshot: root PID 90-ийн төлөв T болсон.

SIGSTOP өгсний дараа PID 90 шар өнгөөр дүрслэгдэв. SIGCONT өгч үргэлжлүүлэн, workload-ийг дуусгахад үүсгэсэн процесс, thread-үүд цэвэрлэгдсэн. Хязгаарласан snapshot-д `truncated=1` тэмдэг ил тод гарсан.

Хүснэгт 8: Хэмжилтүүдийн харьцуулалт

Үе шат	Үр дүн
Created	Root-ийн дэд мод: 5 process + 2 thread.
Stopped	Root-ийн state=T.
After	Workload-ийн task үлдээгүй.
Capped	16 мөр; snapshot дутууг тэмдэглэсэн.
Unloaded	Proc файл устсан.

9. Хэлэлцүүлэг

Граф болон хүснэгт нь kernel-ийн PID, parent, төлөвийн мэдээлэлтэй тохирч байв. Тестээр process-thread ялгалт, DFS дараалал, STOP төлөв, буруу параметр болон модулийн цэвэрлэгээг шалгасан. RCU нь санах ойн аюулгүй уншилтыг хангах боловч бүх системийн нэг агшны атомик зураг батлахгүй. Процесс үүсэх, дуусах үед parent олдохгүй бол root-д холбоод `orphan=1` гэж тэмдэглэнэ. `comm` нь богино task name; бүтэн командын мөр биш. Том системд `max_tasks`-ийг өсгөх шаардлагатай. Kernel ажиллагааг QEMU initramfs орчинд шалгасан. Тайлангийн UI зураг нь тэндээс авсан snapshot-ийн дүрслэл юм. Хадгалсан snapshot нь өмнөх хэмжилтийг давтан үзэхэд зориулагдсан; бодит Linux хэмжилтийг `--kernel` горимоор ажиллуулна.

10. Дүгнэлт

Процессын жагсаалтыг LINEAR болон DFS аргаар гаргах kernel module, task graph UI, snapshot хадгалах болон дахин үзэх боломжийг хэрэгжүүлэв. Нийт 35 автомат шалгалт тэнцсэн. Төслөөр Linux-ийн task бүтэц, process-thread холбоо, төлөв болон DFS-ийг бодит өгөгдөлтэй холбон судалсан.

Ашигласан материал

[1] A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*. 10th ed., Wiley, 2018. Chapter 3, Programming Project 3, P-17–P-19.

[2] Linux 6.8. *Using RCU to Protect Read-Mostly Linked Lists*. `https://docs.kernel.org/6.8/RCU/listRCU.html`

[3] Linux 6.8. *The seq_file Interface*. `https://docs.kernel.org/6.8/filesystems/seq_file.html`

[4] Linux 6.8. *Building External Modules*. `https://docs.kernel.org/6.8/kbuild/modules.html`

Код, тест, баримт бичгийг боловсруулахад ChatGPT/Codex-ийн туслалцаа ашигласан.